

RELATÓRIO DE BANCO DE DADOS

ALUNO: Guilherme Sousa de Sá

CURSO: aprendizagem industrial em programador de sistemas da informação

1. O QUE É UM BANCO DE DADOS?

Um banco de dados é um sistema utilizado para armazenar, organizar, consultar, alterar e controlar dados. Hoje em dia o banco de dados é muito importante para poder gerenciar a maioria dos sistemas utilizados hoje em dia, como lojas, bancos, empresas entre outros.

Durante as aulas foram estudados alguns conceitos sobre banco de dados, utilizando principalmente a linguagem SQL e utilizando o mySQL que é o sistema gerenciador de banco de dados.

2. QUAL BANCO DE DADOS FOI UTILIZADO

O banco de dados utilizado nos projetos durante as aulas foi o MySQL.

O MySQL é um Sistema Gerenciador de Banco de Dados Relacional, também conhecido por SGBD. Ele utiliza a linguagem SQL para manipular e construir dados nas tabelas.

3. POR QUE USAR O MYSQL?

O MySQL é uma boa opção pois ele é um banco de dados relacional, assim podendo relacionar as tabelas e as colunas de diferentes tabelas dentro do banco de dados.

Outra vantagem é a utilização da linguagem SQL. Assim, dentro do banco de dados é possível criar:

- Inserir, alterar e excluir dados;
- Consultar informações dentro das tabelas;
- Criar relacionamentos entre as tabelas;
- Criar views;
- Criar procedures;
- Criar triggers;
- Criar tabelas temporárias.

4. PROCEDURES

As procedures são recursos usados dentro do banco de dados para guardar um conjunto de comandos SQL que realizam uma determinada tarefa. Elas funcionam como uma espécie de rotina que fica salva no banco e pode ser executada sempre que for necessário.

Por exemplo, uma procedure pode ser criada para cadastrar um novo registro, atualizar alguma informação ou excluir um dado. Isso facilita bastante o trabalho, porque não é necessário escrever todos os comandos novamente toda vez que uma operação precisar ser realizada.

Outra vantagem das procedures é que elas ajudam a deixar o banco de dados mais organizado. Em vez de deixar vários comandos espalhados pelo sistema, podemos reunir uma determinada operação dentro de uma procedure e apenas chamá-la quando for necessário.

As procedures também podem receber informações como parâmetros. Dessa forma, é possível passar diferentes valores para a procedure cada vez que ela for executada. Isso faz com que uma mesma procedure possa ser usada várias vezes para diferentes registros.

exemplo de procedure:

```
DELIMITER //
```

```
CREATE PROCEDURE produtosMaisVendidos()
```

```
BEGIN
```

```
    SELECT
```

```
        p.id,
```

```
        p.nome,
```

```
        SUM(pe.quantidade) AS total_vendido
```

```
    FROM produtos p
```

```
    INNER JOIN pedidos pe ON p.id = pe.id_produto
```

```
    GROUP BY p.id, p.nome
```

```
    ORDER BY total_vendido DESC;
```

```
END //
```

```
DELIMITER ;
```

5.TABELAS

As tabelas são uma das partes mais importantes de um banco de dados, porque são nelas que as informações ficam armazenadas. Elas organizam os dados de uma forma que facilita bastante a sua utilização e manutenção.

As tabelas são formadas por colunas e linhas. As colunas representam os tipos de informações que serão armazenadas, enquanto as linhas representam os registros. Por exemplo, em uma tabela de clientes, podemos ter colunas para nome, telefone, CPF, endereço entre outros.

As tabelas também podem possuir uma chave primária, que serve para identificar cada registro de maneira única. Além disso, algumas tabelas possuem chaves estrangeiras, que permitem criar uma ligação entre duas ou mais tabelas.

Essas ligações são importantes porque nem todas as informações precisam ficar em uma única tabela. Podemos separar os dados em diferentes tabelas e depois relacioná-los quando for necessário. Isso ajuda a evitar informações repetidas e deixa o banco mais organizado.

Nas tabelas também podem ser realizadas as principais operações de um banco de dados, como inserir, consultar, alterar e excluir informações.

ex:

Colunas: Adicionar Remover Mover para cima Mover para baixo											
#	Nome	Tipo de dados	Tamanho/lte...	Unsign...	Permitir...	Zerofill	Padrão	Comentário	Colaço	Expressão	Virtualidade
1	id	INT		☐	☐	☐	AUTO_INCREMENT...				
2	nome	VARCHAR	100	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		
3	cnpi_cpf	VARCHAR	18	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		
4	rua	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_general_ci		
5	numero	VARCHAR	50	☐	☒	☐	NULL		utf8mb4_general_ci		
6	bairro	VARCHAR	100	☐	☒	☐	NULL		utf8mb4_general_ci		
7	cep	VARCHAR	10	☐	☒	☐	NULL		utf8mb4_general_ci		
8	referencia	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_general_ci		
9	telefone	VARCHAR	100	☐	☐	☐	Nenhum padrão		utf8mb4_general_ci		

6.VIEWS

As views são usadas principalmente para facilitar a consulta e a visualização das informações que estão armazenadas no banco de dados. Elas são criadas a partir de comandos SQL e podem utilizar informações de uma ou várias tabelas.

Uma view funciona como uma consulta que fica salva no banco. Isso significa que, quando precisamos fazer uma consulta mais complexa várias vezes, não precisamos ficar escrevendo todo o código novamente. Podemos criar uma view e depois consultar essa view de uma maneira muito mais simples.

Por exemplo, podemos ter uma view que mostre apenas determinados produtos, uma que organize informações de vendas ou outra que faça um cálculo usando dados de diferentes tabelas. Dessa maneira, as informações ficam mais fáceis de entender e analisar.

Uma coisa importante é que a view normalmente não armazena uma nova cópia dos dados. Ela apenas guarda a consulta que foi criada. Quando a view é acessada, ela busca as informações nas tabelas relacionadas e apresenta o resultado.

As views são bastante úteis principalmente quando existem consultas grandes ou complicadas. Elas deixam o trabalho mais simples, pois o usuário pode consultar informações de uma maneira mais direta, sem precisar conhecer todos os comandos utilizados para montar aquela consulta.

EX:

```
CREATE VIEW fornecedoresMaisVendas AS
```

```
SELECT
```

```
    f.nome AS fornecedor,
```

```
    COUNT(c.id) AS total_vendas
```

```
FROM compras c
```

```
JOIN fornecedores f
```

```
    ON c.id_fornecedor = f.id
```

```
GROUP BY f.id, f.nome
```

```
ORDER BY COUNT(c.id) DESC;
```

7.TABELAS TEMPORÁRIAS

As tabelas temporárias são tabelas criadas dentro do banco de dados para armazenar informações durante um determinado período de tempo. Elas são diferentes das tabelas comuns porque não têm como objetivo guardar os dados permanentemente. Normalmente, são utilizadas para armazenar informações que precisam ser consultadas ou trabalhadas temporariamente durante a execução de alguma tarefa.

Uma das principais características das tabelas temporárias é que elas ficam disponíveis somente durante a sessão em que foram criadas. Quando essa sessão é encerrada, a tabela temporária é excluída automaticamente pelo banco de dados. Isso significa que não é necessário apagar a tabela manualmente depois que ela não for mais necessária.

Elas podem ser úteis quando precisamos fazer operações intermediárias com os dados. Por exemplo, podemos criar uma tabela temporária para guardar informações que serão utilizadas durante uma consulta, armazenar dados antes de realizar algum processamento ou guardar temporariamente um histórico de alterações.

As tabelas temporárias também ajudam a organizar operações mais complexas. Em vez de realizar várias operações diretamente nas tabelas principais, podemos utilizar uma tabela temporária para guardar os resultados de uma etapa e depois utilizar essas informações em outra operação.

EX:

Colunas: Adicionar Remover Mover para cima Mover para baixo											
#	Nome	Tipo de dados	Tamanho/lte...	Unsign...	Permitir...	Zerofill	Padrão	Comentário	Colaço	Expressão	Virtualidade
1	id_historico	INT		☐	☐	☐	AUTO_INCREMENT				
2	produto_id	INT		☐	☒	☐	NULL				
3	nome_antigo	VARCHAR	255	☐	☒	☐	NULL		utf8mb4_0900_ai_ci		
4	preco_antigo	DECIMAL	10,2	☐	☒	☐	NULL				
5	estoque_antigo	FLOAT		☐	☒	☐	NULL				
6	data_alteracao	DATETIME		☐	☒	☐	now()				
7	usuario_altera...	VARCHAR	100	☐	☒	☐	NULL		utf8mb4_0900_ai_ci		